

NUMERICAL ANALYSIS GUIDED LAB 2

ROB TOMPKINS

Note, all the files associated with Guided Lab 2 are checked into the :

<https://github.com/chtompki/MathSpring2026/tree/main/NumericalAnalysis/GuidedLab2>

Exercise 1. We've added the code files to github at the following links: `gram_schmidt.m` `problem1.m`

(a) We went about completing this problem by writing the following code:

```
function Q = gram_schmidt( X )
    % Q = gram_schmidt( X )
    % X is an d by n matrix
    % Q is an d by r matrix with 1 <= r <= n that is linearly independent
    % computes the Gram-Schmidt left decomposition matrix Q
    % By Rob Tompkins, 20260215
    [d,n] = size(X);
    m = min(d,n);
    R = zeros(m,n);
    Q = zeros(d,m);
    for i = 1:m
        v = X(:,i);
        for j = 1:i-1
            R(j,i) = Q(:,j)'\*v;
            v = v-R(j,i)*Q(:,j);
        end
        R(i,i) = norm(v);
        Q(:,i) = v/R(i,i);
    end
    R(:,m+1:n) = Q'\*X(:,m+1:n);
end
```

(b) We have two means by seeing that $X = \begin{bmatrix} 1 & 1 & 1; & 0 & 1 & 1; & 0 & 0 & 1 \end{bmatrix}$ the Gram-Schmidt algorithm should yield I_3 , the identity matrix on $\mathbb{R}^3$. The first way to notice is that it is an upper triangular matrix with no repeated columns. The other is that if we run the algorithm on the matrix X in matlab (see `problem1.m` for the matlab output) we get

$$Q1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

(c) Similar to problem (b) it is easy to see that $X = \begin{bmatrix} 1 & 1 & 1; & 1 & 1 & 0; & 1 & 0 & 0 \end{bmatrix}$ has determinant -1, and we can see that $\text{span}(X) = \text{span}\{e_1, e_2, e_3\}$ where $e_i \in \mathbb{R}^3$ is the vector that is all zeros except for 1 in the i^{th} -place. Furthermore, we ran our Gram-Schmidt process on X and calculated the norm over the column vectors and we got 1.

(d) We ran the Gram-Schmidt algorithm on the matrix $X = \begin{bmatrix} 2 & -1 & 0; & 0 & -1 & 2; & -1 & 0 & 0; -12- \\ 1 \end{bmatrix}$ to get the following Q matrix

$$Q = \begin{bmatrix} 0.816496580927726 & 0.182574185835056 & 0.000000000000000 \\ 0 & -0.547722557505166 & 0.816496580927726 \\ -0.408248290463863 & -0.365148371670111 & -0.408248290463863 \\ -0.408248290463863 & 0.730296743340221 & 0.408248290463863 \end{bmatrix}.$$

If you look at the problem1.m file that we've linked to you'll see that each of the columns has length 1.

(e) It's quite impossible for Q to be orthogonal because $Q^T Q \in M_{3 \times 3}(\mathbb{R})$ and $Q Q^T \in M_{4 \times 4}(\mathbb{R})$, and $Q Q^T$ has non-zero entries off the diagonal.

Exercise 2. (a) For our “gs_factor” algorithm (given in the file gs_factor.m) that does the full decomposition $X = QR$ using the algorithm above we simply need to add R to the return values of the function. We can simply do this in the following manner:

```
function [Q, R] = gs_factor( X )
    % Q = gram_schmidt( X )
    % X is an m by n matrix
    % Q is an m by r matrix with 1 <= r <= n that is linearly independent
    % R is an n by n upper triangular matrix
    % computes the Gram-Schmidt left decomposition matrix X=QR
    % By Rob Tompkins, 20260215
    [d,n] = size(X);
    m = min(d,n);
    R = zeros(m,n);
    Q = zeros(d,m);
    for i = 1:m
        v = X(:,i);
        for j = 1:i-1
            R(j,i) = Q(:,j)'*v;
            v = v-R(j,i)*Q(:,j);
        end
        R(i,i) = norm(v);
        Q(:,i) = v/R(i,i);
    end
    R(:,m+1:n) = Q'*X(:,m+1:n);
end
```

(b) We used our matlab code for the following computation of the decomposition of the matrix in to $X = QR$:

$$X = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

$$Q = \begin{bmatrix} 0.577350269189626 & 0.408248290463863 & 0.707106781186547 \\ 0.577350269189626 & 0.408248290463863 & -0.707106781186548 \\ 0.577350269189626 & -0.816496580927726 & 0 \end{bmatrix}$$

$$R = \begin{bmatrix} 1.732050807568877 & 1.154700538379252 & 0.577350269189626 \\ 0 & 0.816496580927726 & 0.408248290463863 \\ 0 & 0 & 0.707106781186547 \end{bmatrix}$$

(c) We let $A = \text{rand}(100)$ in Matlab, and we ran $[Q1, R1] = \text{gs_factor}(A)$. Then we ran the subsequent calculations for the sake of verifying various qualities of $Q1$ and $R1$:

```
[Q,R] = gs_factor([ 1 1 1; 1 1 0; 1 0 0 ])
Q*R
```

```
A = rand(100,100);
det(A)
[Q1,R1] = gs_factor(A);
det(Q1*R1)
norm(A)
norm(Q1*R1)
```

```
norm(Q1'*Q1 - eye(100))
norm(Q1*Q1' - eye(100))
norm(R - triu(R))
```

The results all came out as we would expect with $Q1$ being orthogonal, $R1$ being upper triangular, and $X = QR$. The calculations and answers can all be seen in the file problem2.m.

3.

4.